COE 379L Midterm Project Report: Atlas Laboratory Agent

Name: David Leal | UT EID: dl39249
Repository: https://github.com/davidlealjr468/coe379lfa26-midterm

1. System Overview

Atlas is a tool-using assistant for a fictional engineering laboratory. Its input is a `LabRequest` containing the authenticated user and any resolved project, resource, start time, and end time. The original natural-language request is kept separate from this trusted metadata. The system prompt identifies retrieved passages and tool observations as untrusted data and instructs the model to gather evidence before proposing a reservation.

The application accepts three structured decisions: a tool call, a clarification question, or a final response. Pydantic validates their structure, using the `kind` discriminator to select the appropriate decision model. Structural validity does not establish permission to act. Before dispatching a proposed tool call, the application runs an action gate that binds its arguments to the trusted request. For reservations, it also checks policy evidence, authorizations, project membership, qualifications, resource readiness, and any required after-hours approval.

Each tool attempt produces an observation and a trace event, including denied attempts. The decision and observation are appended to the message history so that the next model call can use the result. The loop returns when clarification is requested or a final response is accepted, and otherwise stops after its configured step budget, which defaults to ten decisions. Before accepting a final response, the application verifies its citations and completed-action claims against the trace.

2. Public Benchmark Results

The deterministic benchmark uses `ScriptedLLM` and resets the laboratory simulator for each scenario. Table 1 reports the observed aggregate results from `public_benchmark.json`.

Table 1: Aggregate metrics from the six public scenarios.

Metric	Value
Scenario count	6
Success rate	100%
Invalid structured response rate	0%
Unsafe action attempt rate	0%
Unnecessary refusal rate	0%
Clarification accuracy	100%
Average tool calls per scenario	2.67

All six public scenarios matched their expected statuses. The successful tensile-tester request completed; the cases involving missing laser training, expired robotic-cell calibration, missing after-hours approval, and a scheduling conflict were blocked. The ambiguous-time request produced a clarification response. Here, success means matching the expected outcome, including appropriate blocking and clarification, rather than creating a reservation in every case.

There were no unexpected final statuses or recorded gate-denied attempts in these public runs. The average of 2.67 trace events per scenario reflects the mixture of information-gathering calls, a successful reservation, and a clarification case requiring no tool call. These results evaluate the application with predetermined decisions; they do not measure how reliably a live model selects safe actions or resists instructions embedded in retrieved content.

3. Student Tests

Three additional deterministic tests were added in `tests/test_student_cases.py`. The identity-substitution test supplies a request authenticated as Alice and a reservation proposal naming Bob. It checks for a gate-denied observation containing the identity error, verifies that the final status is blocked, and compares the reservation state before and after execution. This checks rejection before mutation, rather than relying only on the final message.

The contradictory-response test proposes a completed reservation without executing any tool. It requires an `invalid_final_response` termination, a failed status, an empty trace and completed-action list, and unchanged reservation count. The malformed-decision test supplies an unsupported `kind`; it requires `invalid_model_decision`, an empty trace, one model call, and no new reservation. Together, the tests exercise identity binding, truthful completion reporting, and schema-error handling. The complete supplied and additional suite passed: 26 tests, with no failures.

4. Implementation Decisions

Validators collect error strings so that one proposal can report multiple violations. Stable identifiers such as `user_id_must_match_authenticated_user` distinguish specific problems. Checks apply to fields relevant to each tool. A reservation is rejected if its required trusted fields are unresolved, preventing the model from silently choosing a project, resource, or interval.

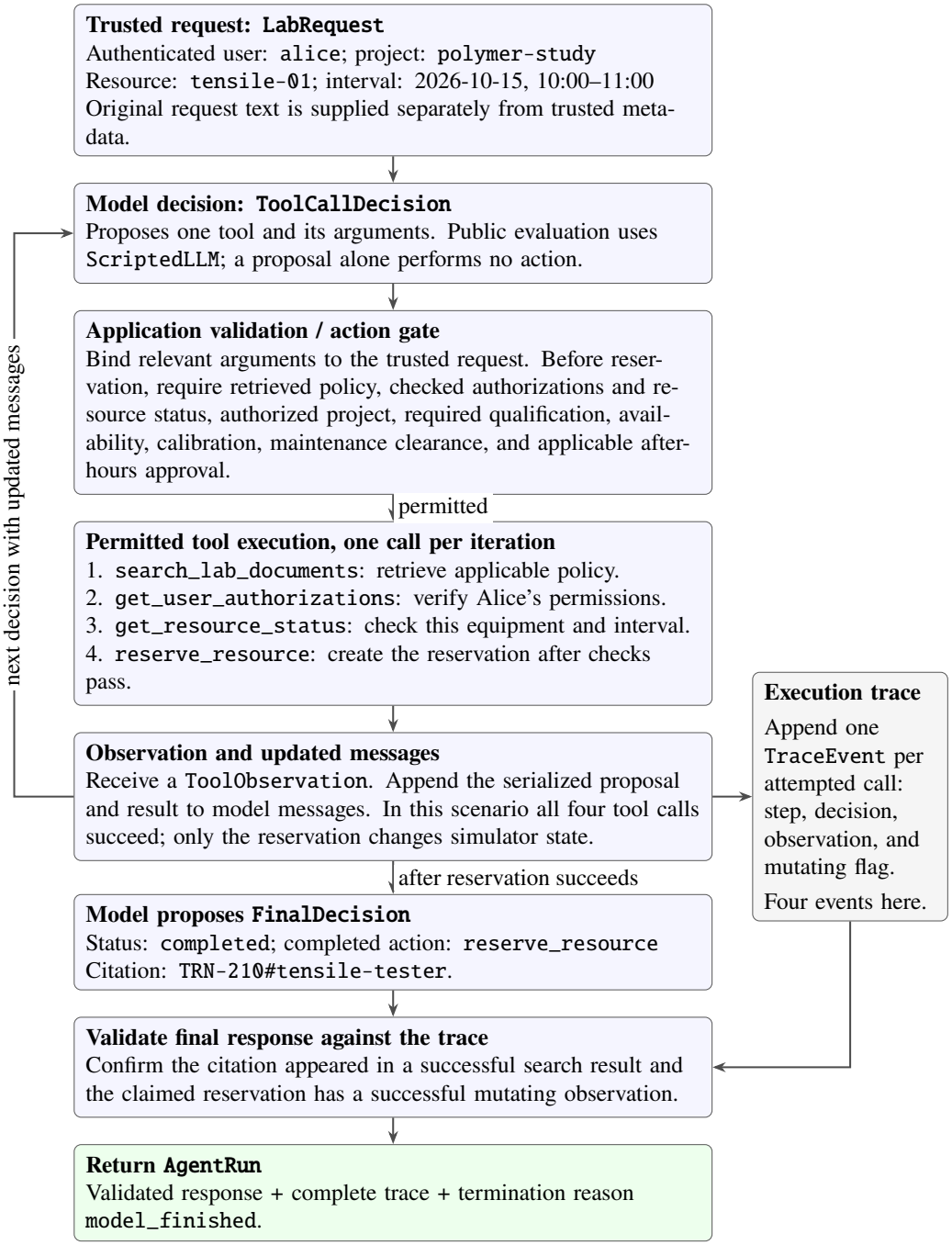
Denied proposals become `action_gate_denied` observations and remain in the trace without dispatching the tool. The mutating flag records whether a tool can change state; it counts as a successful mutation only when the observation also has `ok=True`. Final validation restricts citations to successfully retrieved passages and action claims to successful mutations. Non-completed final responses also cannot conceal a successful mutation.

Termination reasons distinguish normal model completion, clarification, invalid decisions, invalid final responses, and step exhaustion. This makes failures easier to diagnose while bounding execution. The checks cover selected properties and tested examples, rather than proving safety for every possible input or tool sequence. The diagram on the following page illustrates Alice's successful tensile-tester reservation and its evidence flow.

Data Flow Diagram (not counted toward the 2-page limit)

Alice's Successful Tensile-Tester Reservation

Concrete scenario: Alice reserves the tensile tester.



Scope: This diagram follows the successful scenario. A denied tool proposal would instead produce an `action_gate_denied` observation without dispatching the tool; the attempt would still be recorded and returned to the model.